

MLIR NPU Compiler - Lecture 08

SCF/Affine Loop: M/N/K Tile Loop and DMA Scheduling

강의 목표

- SCF와 Affine dialect의 역할을 구분하고 NPU compiler lowering 단계에서 언제 사용하는지 판단한다.
- linalg.matmul을 M/N/K tiled loop nest로 읽고 작성한다.
- NPU SRAM tile footprint, accumulator lifetime, DMA load/store, barrier/wait 순서를 설계한다.
- SCF/Affine loop 예제를 FileCheck regression test 관점으로 검토한다.

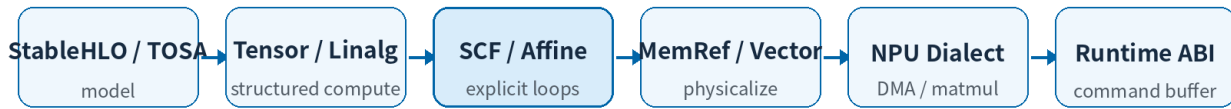
항목	내용
강의 위치	StableHLO/TOSA -> Tensor/Linalg 이후, Bufferization/MemRef/Vector/NPU Dialect 이전의 explicit loop 단계
핵심 산출물	M/N/K tile loop와 DMA 순서 설계 문서 + pseudo MLIR
필요 배경	1~7 강: Operation/Region/Block, mlir-opt/FileCheck, Dialect, StableHLO/TOSA, Linalg-on-Tensors
NPU 관점	loop order, tile shape, SRAM footprint, DMA overlap, accumulator reuse, edge tile policy

1. 왜 SCF/Affine Loop가 중요한가?

Linalg 단계는 structured compute를 높은 추상도로 보존한다. 그러나 NPU에서 실행하려면 compiler는 어느 순서로 tile을 순회할지, 어떤 tile을 SRAM에 올릴지, 언제 DMA를 기다릴지, accumulator를 언제 초기화하고 store할지를 결정해야 한다. 이때 SCF와 Affine은 compute를 explicit loop로 노출하는 중간 단계다.

- SCF는 구조화된 control flow를 표현하며 dynamic bound, runtime condition, command scheduling skeleton에 유리하다.
- Affine은 affine map과 affine loop/access를 통해 정적 분석, dependence analysis, loop transform에 유리하다.
- NPU compiler에서는 Linalg에서 schedule을 결정한 뒤 SCF/Affine으로 lowering하고, 이후 MemRef/Vector/NPU dialect로 physicalization한다.

Lecture 08: SCF / Affine Loop 위치



NPU compiler 질문

- M/N/K loop order는 어떤 reuse를 만드는가?
- tile shape가 SRAM/accumulator/DMA burst에 맞는가?
- DMA와 compute를 overlap할 수 있는가?
- edge tile과 dynamic shape policy는 무엇인가?

2. SCF Dialect 핵심

SCF 는 if/for 같은 structured control flow 를 표현한다. unstructured goto 형태가 아니라 Region 과 Block 구조 안에 body 를 담기 때문에 pass 가 loop body 를 안정적으로 분석하고 변환할 수 있다.

scf.for anatomy

```
func.func @scf_for_basic(%A: memref<128xf32>, %B: memref<128xf32>) {
  %c0 = arith.constant 0 : index
  %c128 = arith.constant 128 : index
  %c1 = arith.constant 1 : index
  scf.for %i = %c0 to %c128 step %c1 {
    %a = memref.load %A[%i] : memref<128xf32>
    memref.store %a, %B[%i] : memref<128xf32>
  }
  return
}
```

- lower bound, upper bound, step 은 모두 index-typed SSA value 일 수 있다.
- body 의 induction variable `i` 는 block argument 처럼 동작한다.
- loop-carried value 는 `iter\_args` 와 `scf.yield` 로 표현한다.
- NPU 관점에서 `iter\_args` 는 accumulator state, token, software pipelining state 를 설명할 때 유용하다.

3. Affine Dialect 핵심

Affine dialect 는 loop bound 와 memory access 가 affine expression 으로 표현될 때 강력하다. affine map 은 dimension 과 symbol 을 구분하고 compiler 가 access pattern 과 dependence 를 더 잘 분석할 수 있도록 한다.

- 정적 tile loop, affine load/store, affine.apply 기반 index 계산에 적합하다.
- loop interchange, tiling, dependence check, vectorization legality 판단에 활용 가능하다.
- runtime data-dependent branch, non-affine indexing, complex boundary policy 는 SCF/arith 조합이 더 자연스럽다.

SCF vs Affine Decision Matrix

SCF Dialect

- structured control flow
- dynamic bounds / runtime branch
- scf.for, scf.if, scf.forall
- NPU command scheduling skeleton
- less restrictive, less analyzable

Affine Dialect

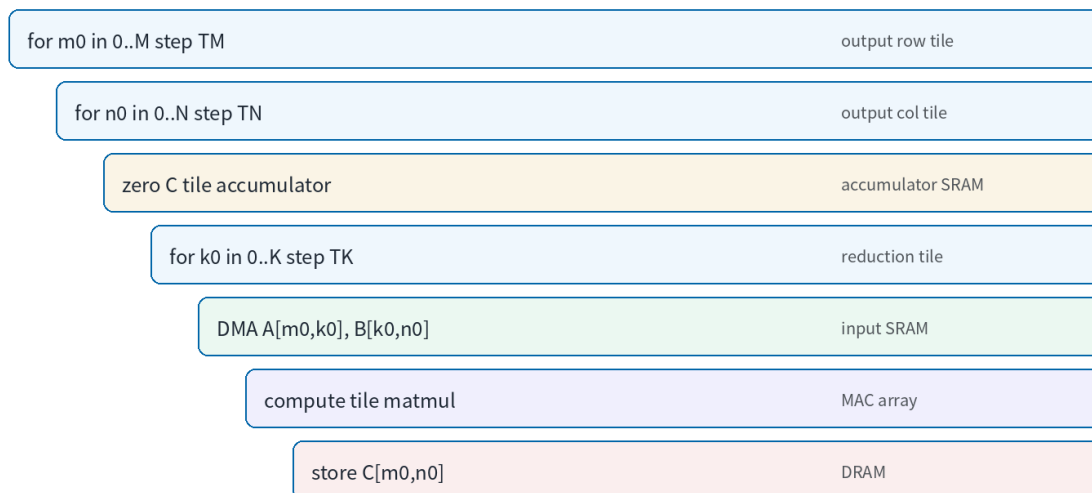
- affine loop/access/maps
- static loop analysis
- affine.for, affine.apply
- tiling/interchange/dependence
- more restrictive, more analyzable

Rule: dynamic scheduling은 SCF, 정적 loop/access 분석은 Affine으로 두는 것이 일반적이다.

4. MatMul 을 M/N/K Tile Loop 로 보기

MatMul 은 $C[m,n] += A[m,k] * B[k,n]$ 구조다. NPU 에서는 output C tile 을 accumulator 에 잡고, inner K tile 을 순회하며 A/B tile 을 SRAM 으로 가져와 MAC array 에 공급하는 방식이 일반적이다.

M/N/K Tiled MatMul Loop Nest



Tile footprint: $A=TM*TK*input_bytes$, $B=TK*TN*input_bytes$, $Acc=TM*TN*acc_bytes$

Loop order determines accumulator lifetime, input tile streaming, and DRAM burst behavior.

SCF pseudo loop

```
scf.for %m0 = %c0 to %M step %TM {
```

```

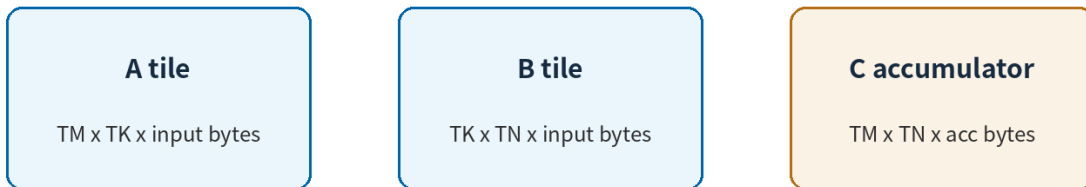
scf.for %n0 = %c0 to %N step %TN {
  "npu.acc.zero"(%m0, %n0, %TM, %TN) : (...) -> ()
  scf.for %k0 = %c0 to %K step %TK {
    "npu.dma.load_a"(%A, %m0, %k0, %TM, %TK) : (...) -> ()
    "npu.dma.load_b"(%B, %k0, %n0, %TK, %TN) : (...) -> ()
    "npu.barrier.wait_dma"() : () -> ()
    "npu.matmul.tile"(%m0, %n0, %k0, %TM, %TN, %TK) : (...) -> ()
  }
  "npu.dma.store_c"(%C, %m0, %n0, %TM, %TN) : (...) -> ()
}
}

```

5. SRAM Footprint 와 Tile Size

Tile size 는 단순 loop parameter 가 아니라 hardware contract 다. SRAM capacity, bank, DMA burst alignment, accumulator width, vector lane, systolic array shape 와 모두 연결된다.

SRAM Tile Footprint



Example: TM=32, TN=64, TK=64, int8 input, int32 accumulator

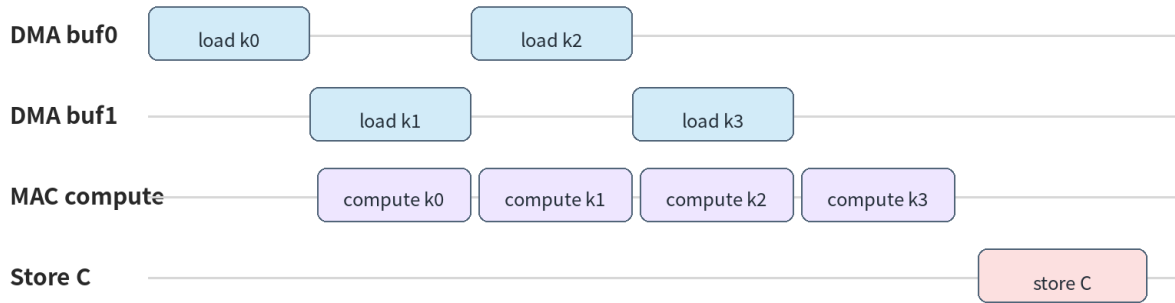
- $A = 32 \times 64 \times 1 = 2 \text{ KB}$
- $B = 64 \times 64 \times 1 = 4 \text{ KB}$
- $\text{Acc} = 32 \times 64 \times 4 = 8 \text{ KB}$
- $\text{Double-buffered A/B} + \text{Acc} = 2 \times (2 + 4) + 8 = 20 \text{ KB}$
- Add alignment/tag/bank overhead before accepting tile.

항목	계산식/정책
A tile	$TM * TK * \text{input bytes}$
B tile	$TK * TN * \text{input bytes}$
Accumulator	$TM * TN * \text{accumulator bytes}$
Double buffer	$2 * (A \text{ tile} + B \text{ tile}) + \text{accumulator}$
Policy	SRAM/bank/alignment 위반 시 retile 또는 fallback path 선택

6. DMA 순서와 Double Buffering

성능 좋은 NPU schedule 은 DMA 와 compute 를 완전히 직렬화하지 않는다. 일반적으로 current buffer 로 compute 를 수행하는 동안 next buffer 에 다음 K tile 을 preload 한다. wait/barrier 는 true dependency 앞에만 둔다.

Double-buffered DMA / Compute Timeline



Scheduling rule: preload next K tile while MAC consumes current tile. Wait only before true consume.

- no compute-before-DMA-complete
- no DMA overwrite-before-consume
- store only after all K tiles accumulated
- edge tile must preserve valid read/write ranges

- DMA start: next A/B tile 을 SRAM buffer 에 비동기 로드한다.
- DMA wait/barrier: compute 가 해당 tile 을 소비하기 직전에 완료를 보장한다.
- Buffer hazard: MAC 이 읽는 buffer 를 DMA 가 덮어쓰면 안 된다.
- Store hazard: accumulator result 가 완성되기 전 DRAM store 가 시작되면 안 된다.

7. Edge Tile 과 Dynamic Shape

M/N/K 가 tile size 의 배수가 아닐 때 compiler 는 effective tile size 또는 mask/padding/fallback policy 를 명시해야 한다. 이 정책은 correctness 와 performance 에 직접 영향을 준다.

- effective tile: $\text{tm_eff} = \min(\text{TM}, \text{M} - \text{m0})$ 형태로 실제 유효 범위를 계산한다.
- mask: hardware 가 partial tile mask 를 지원하면 compute kernel 에서 mask 를 적용한다.
- padding: input 을 tile boundary 까지 pad 하고 output writeback 에서 valid region 만 store 한다.
- fallback: 작은 edge tile 은 scalar/vector CPU 또는 다른 NPU micro-kernel 로 처리한다.

8. 실습 구성

1. Example 01 에서 `scf.for` 의 bound, step, induction variable, body operation 을 annotation 한다.
2. Example 02 에서 M/N/K tile loop 를 읽고 accumulator lifetime 을 표시한다.
3. Example 03 에서 affine loop 가 어떤 access pattern 을 표현하는지 분석한다.
4. Example 04 를 기반으로 double-buffered DMA sequence table 을 작성한다.
5. Tile footprint calculator 로 tile 후보 3 개를 비교하고 reject/accept 정책을 쓴다.

9. 강의 중 토론 질문

- M-major vs N-major loop order 중 어느 쪽이 DRAM burst, B reuse, output store 에 유리한가?
- K tile 을 크게 하면 arithmetic intensity 는 좋아지지만 SRAM 과 latency hiding 에는 어떤 trade-off 가 생기는가?
- DMA wait 를 각 tile 마다 넣는 방식과 token dependency 기반으로 넘기는 방식의 장단점은 무엇인가?
- Affine 으로 표현하면 좋은 loop 와 SCF 로 남기는 것이 좋은 loop 를 어떻게 나눌 것인가?

10. Homework

- MatMul shape `M=256, N=192, K=384`, tile 후보 `(32,64,64)`, `(64,64,64)`, `(32,128,64)` 에 대해 SRAM footprint 와 loop trip count 를 계산한다.
- 각 후보에 대해 A reuse, B reuse, accumulator footprint, DMA burst friendliness 를 비교한다.
- 가장 좋은 후보를 하나 선택하고, SCF pseudo IR 과 DMA schedule table 을 작성한다.
- FileCheck 로 확인할 loop structure 및 npu pseudo op ordering check 를 5 줄 이상 작성한다.

References

- MLIR SCF Dialect: <https://mlir.llvm.org/docs/Dialects/SCFDialect/>
- MLIR Affine Dialect: <https://mlir.llvm.org/docs/Dialects/Affine/>
- MLIR MemRef Dialect: <https://mlir.llvm.org/docs/Dialects/MemRef/>
- MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>

주의: 일부 예제의 `npu.\*` operation 은 18 강 custom NPU dialect 설계를 미리 보여주기 위한 pseudo IR 입니다.